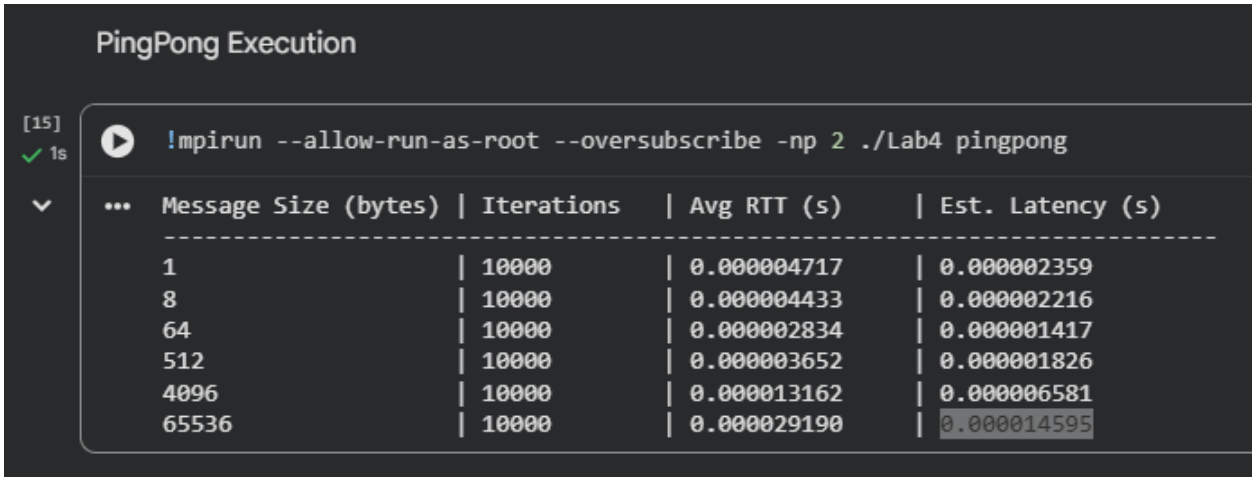
Intro to Parallel Processes Lab 4 Answers

Name: Thirunavukarasu MuthuValliappan
Student No: 3785346

Ping Pong Timing Table:

Message Size (bytes)	Iterations	Avg RTT (s)	Est. Latency (s)
1	10000	0.000004717	0.000002359
8	10000	0.000004433	0.000002216
64	10000	0.000002834	0.000001417
512	10000	0.000003652	0.000001826
4096	10000	0.000013162	0.000006581
655536	10000	0.000029190	0.000014595



Answers:

1. How is message size controlled?

The message size is controlled by allocating a raw byte buffer of a specific length using malloc(msg_size) and passing that exact size as the count argument to MPI_Send or MPI_Recv. By using the MPI_BYTE datatype, the program ensures that MPI sends exactly the requested number of elements, where each element is one byte.

2. How is correctness verified?

Correctness is verified by deriving a message tag from the iteration number **i** using modulo arithmetic. Instead of inspecting the buffer's payload, the receiver validates the exchange by checking status.MPI_TAG against the expected iteration tag to ensure messages are synchronized and matching correctly.

3. Why/how does your implementation avoid deadlock?

In the ping-pong exchange, deadlock is prevented through strict ordering: rank 0 is programmed to send then receive, while rank 1 is programmed to receive then send. This sequential flow ensures that one process is always ready to receive while the other is sending, avoiding a state where both processes are stuck waiting on each other. In the ring topology, we use MPI_Sendrecv, which combines the send and receive operations into a single atomic call. This allows all processes to simultaneously initiate a send to their right neighbor and a receive from their left neighbor, effectively breaking the circular dependency that would cause a "naïve" blocking pattern to hang.

---

1. Why is timing performed on rank 0 only?

Timing is performed on rank 0 only to ensure consistency and avoid the complexities of clock synchronization between different processes. Since the round-trip time (RTT) measures the full duration from the initial send at rank 0 to the final receipt of the reply, rank 0's local clock can accurately capture the entire communication cycle. Measuring on multiple ranks would introduce "clock drift" or offsets, making it difficult to combine results without a perfectly synchronized global clock.

2. Why is RTT/2 only an approximation of one-way latency?

Dividing the RTT by two assumes the communication path is perfectly symmetrical. In reality, one-way latency is an approximation because it doesn't account for directional imbalances like asymmetric network congestion, OS scheduling variations, or differences in how MPI handles sending versus receiving on each node.

3. Why can this deadlock if all processes execute it simultaneously? / Why does a pattern of sequential MPI_Send followed by MPI_Recv cause a deadlock in a ring topology when all processes execute it at the same time?

This scenario causes a deadlock because MPI_Send is a blocking routine that may not return until the message is safely stored or received. If every process in the ring starts with MPI_Send simultaneously, they all wait for their respective neighbor to call MPI_Recv. However, since every neighbor is also stuck at their own MPI_Send, no process ever reaches the MPI_Recv call, creating a circular dependency where everyone is waiting and no one can proceed.

Confirmation of correct ring behaviour:

Correct ring behavior is confirmed when each process receives the rank of its left neighbor as the recv_value. The below screenshot denotes the program execution with 8 processors.

